Task-5:- Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to outline the functional and non-functional requirements for deploying a 3-tier web application on Amazon Web Services (AWS) and establishing a CI/CD pipeline. The intended audience includes the Cloud Engineering team, DevOps engineers, and project stakeholders.

1.2 Scope

- **In-Scope:** Infrastructure provisioning (VPC, EC2/S3, RDS), networking configuration, GitHub Actions/AWS CodePipeline setup, and end-to-end connectivity testing.
- **Out-of-Scope:** Modification of the application's core source code, Domain Name registration (using raw IPs/DNS), and long-term database maintenance.

1.3 Definitions, Acronyms, and Abbreviations

- **FE/BE:** Frontend / Backend.
- **3-Tier:** Architecture consisting of Presentation, Application, and Data layers.
- **CI/CD:** Continuous Integration and Continuous Deployment.
- **RDS:** Relational Database Service.
- **ALB:** Application Load Balancer.

1.4 References

- GitHub Repository: `https://github.com/pushkar-iamops/fe-be`
- AWS Well-Architected Framework documentation.

1.5 Overview

This document transitions from project objectives to technical use cases, concluding with risk analysis and security requirements to ensure a robust deployment.

---

2. Project Overview

2.1 Project Objectives

- Migrate the `fe-be` app to a scalable AWS environment.
- Automate deployment to eliminate manual updates.

2.2 Business Goals

- Reduce deployment downtime through automation.
- Ensure data integrity and high availability of the application.

2.3 Background

The cloud team requires a standardized 3-tier setup to replace legacy or local hosting methods, ensuring the app can handle production-level traffic and security standards.

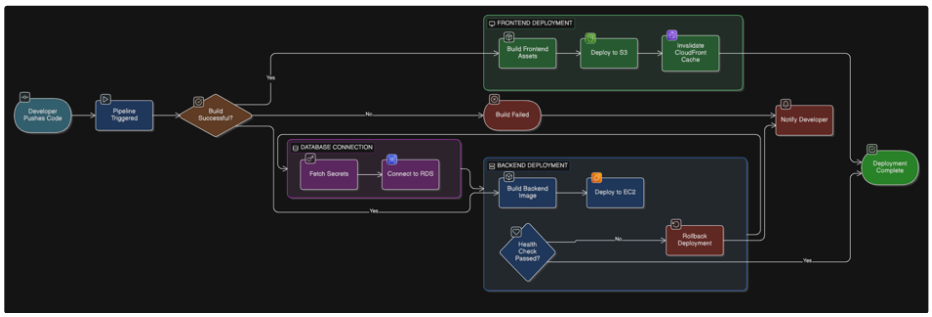
2.4 Constraints

- **Timeline:** Task must be completed within the assigned sprint.
- **Platform:** Limited strictly to AWS services.
- **Access:** Deployment must be triggered exclusively via GitHub pushes.

---

3. Functional Requirements

3.1 Use Case Diagram



3.2 Use Cases

3.2.1 Use Case 1: Automated Code Deployment

- **Description:** The system automatically updates the live environment when code is pushed to GitHub.
- **Actors:** Developer (Pushkar), GitHub Actions/AWS CodePipeline.
- **Preconditions:** AWS credentials stored in GitHub Secrets; Code pushed to `main` branch.
- **Postconditions:** Frontend and Backend services reflect the latest commit.
- **Flow of Events:**
  a. Developer pushes code to the repository.
  b. GitHub Webhook triggers the CI/CD pipeline.
  c. Pipeline builds the Docker images or build artifacts.
  d. Pipeline deploys artifacts to S3 (FE) and EC2/ECS (BE).
- **Alternative Flows:**
  - **Failure:** If build fails, the pipeline stops and notifies the developer via logs.

---

4. Non-Functional Requirements

4.1 Performance Requirements

- **Latency:** Backend API response time should be $<200ms$ under normal load.
- **Build Time:** The automation pipeline should complete in under 5 minutes.

4.2 Security Requirements

- **Isolation:** Database must reside in a private subnet with no public IP.
- **Encryption:** Use SSL (HTTPS) for the frontend via CloudFront/ACM.
- **Secrets:** No hardcoded DB credentials; use AWS Secrets Manager or Environment Variables.

4.3 Usability Requirements

- The application must be accessible via a public-facing URL (ALB DNS or CloudFront URL).

4.4 Reliability Requirements

- **Uptime:** Target 99.9% availability using Multi-AZ deployment for the database.
- **Recovery:** Automated health checks to restart failing Backend instances.

---

5. Assumptions and Dependencies

5.1 Assumptions

- The provided GitHub code is functional and passes local build tests.
- The user has sufficient AWS service quotas for RDS and EC2.

5.2 Dependencies

- **GitHub Availability:** The automation depends on GitHub's uptime.
- **AWS IAM:** Correct permissions must be granted to the deployment user.

---

6. Risk Analysis

6.1 Risk Identification

- **R1:** Database connection failure due to incorrect Security Group rules.
- **R2:** Exceeding AWS Free Tier budget.

6.2 Risk Mitigation

- **M1:** Implement "Least Privilege" security groups; allow only BE $\rightarrow$ DB traffic on port 5432/3306.
- **M2:** Set up AWS Billing Alarms.

---

7. Glossary

- **VPC:** Virtual Private Cloud (Your private network in AWS).
- **Subnet:** A range of IP addresses in your VPC.
- **IAM:** Identity and Access Management.

---

8. Appendices

- Architecture Diagram (to be attached in Technical Doc).
- Environment Variable List (`DB_HOST` , `DB_USER` , `PORT`).

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____